

Koji algoritam je šampion? Odabir najboljeg modela za naš problem

Četvrtak, 09:17 AM – Tim na Google Colab chatu, #ml-projekat-reviews-v.2

Marc: Ljudi, stigli smo do dela kada naš projekat više nije igralište. Sad biramo najboljeg igrača! Koji algoritam će najbolje prepoznati sentiment korisničkih recenzija?

Joakim: I vreme je za malo takmičenje! Da vidimo, da li „stara škola” Logistic Regression i Naive Bayes i dalje drže prednost, ili će Random Forest i SVM napraviti iznenađenje?

Alex: Meni je ovo prvi put da učestvujem u ovakvom ozbiljnom nadmetanju modela. Baš me zanima kako izgleda pravo takmičenje između algoritama, pogotovo na većim podacima!

Mašinsko učenje nije jedno rešenje – postoji više algoritama koji rešavaju isti zadatak na različite načine.

Sada je trenutak da postavimo ključno pitanje: Koji algoritam mašinskog učenja će najbolje obaviti posao klasifikacije sentimenta recenzija?

Šta radimo u ovoj lekciji?

Kroz ovu lekciju:

- delimo podatke na skupove za treniranje i testiranje – tako da poređenje bude fer i transparentno;
- treniramo nekoliko različitih algoritama na istim podacima;
- evaluiramo svaki model koristeći ključne metrike: preciznost, odziv, F1 skor i ukupnu tačnost;
- analiziramo rezultate i diskutujemo o tome koji model daje najbolji balans između performansi i jednostavnosti primene.

Ne zaboravi, u realnom svetu često nije dovoljno da model radi – tražimo rešenje koje je i robustno i skalabilno i lako za održavanje.

Spremi se, ulazimo u arenu modela!

Na kraju ove lekcije, bićeš u stanju da samostalno porediš i biraš najbolji model za različite zadatke u mašinskom učenju.

Pravi start: Kako i zašto delimo podatke?

10:40 AM – Tim na Colab chatu

Alex: Pre nego što pustimo algoritme u arenu, vreme je da napravimo fer teren. Da, pričam o onoj famoznoj podeli na trening i test setove!

Joakim: I neka više niko ne pita zašto ne treniramo na svim podacima odjednom!

Marc: Kad treniraš na svemu, nemaš realnih podataka na kojima možeš da utvrdiš njegove performanse. A svakako nije dobro prvi put tenirati model u produkciji.

Alex: Ovo nije prvi put da radimo podelu – ali sad radimo profesionalnije i sa više inputa!

Pre nego što model bude u stanju da nešto nauči, potrebno je da razdvojimo svoj skup podataka na:

- x = ulazne karakteristike (skup za treniranje) – ono što model koristi kao osnovu za učenje (review_title, review_text, review_length);
- y = ciljnu promenljivu (skup za testiranje) – ono što pokušavamo da pogodimo (sentiment).

Zato biramo 80% podataka za treniranje, a 20% za testiranje (i pazimo da raspodela sentimenta ostane ista u oba dela – zato koristimo stratify).

Evo kako ćemo da obavimo podelu podataka na ulazne karakteristike i ciljnu promenljivu:

```
1 # Features and label
2 X = df[["review_title", "review_text", "review_length"]]
3 y = df["sentiment"]
```

Zatim ćemo podeliti podatke na skup za treniranje (80%) i skup za testiranje (20%) kako bismo mogli da validiramo koliko model dobro radi na podacima koje nikada ranije nije video:

```
1 from sklearn.model_selection import train_test_split
2
3 # Train-test split
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42, stratify=y
6 )
```

U ovim koracima nije bilo ničeg novog. Jedina razlika u odnosu na ono što smo do sada videli jeste to što sada imamo tri ulazne karakteristike na osnovu kojih će model biti treniran.

Pogledajmo zajedno kako se obavlja podela podataka na ulazne/izlazne i trening/test setove:

<https://www.youtube.com/watch?v=MrwLWzqFyg0>

Video: Podela podataka na ulazne/izlazne i trening/test setove
Autor: ITAcademy

Mini-aktivnost: Kako biste vi podelili podatke?

Probaj da promeniš odnos (npr. 70/30 ili 60/40) ili ukloniš stratify – šta se dešava sa raspodelom sentimenta u train i test setovima?

Zašto je važno da podela bude reprezentativna za sve klase?

Od sirovih reči do brojeva: Priprema podataka za mašinsko učenje

Google Colab chat, #ml-projekat-reviews-v.2

Alex: E, ekipo, došao je onaj trenutak kad moramo da „prevedemo” svoje podatke tako da ih i algoritmi razumeju. Sirovi tekst? Lepo zvuči, ali za model je to isto što i pesma na stranom jeziku bez titla!

Joakim: Dakle, vreme je za TF-IDF magiju? Znači, reči i slova postaju brojevi!

Mašinski modeli ne mogu da razumeju tekstualne podatke u sirovom obliku. Zato je potrebno da tekstualne kolone pretvorimo u numeričke vektore.

Sada ćemo otići i korak dalje, pa ćemo da obradimo i podatke unutar numeričke kolone `review_length`. Iako je vrednosti unutar takve kolone moguće direktno proslediti modelu (zato što su brojevi), u nastavku ćemo obaviti normalizaciju vrednosti u takvoj koloni i na taj način olakšati modelu interpretaciju njenih vrednosti.

Za sve to ćemo da koristimo:

- `TfidfVectorizer` za kolone `review_title` i `review_text` – ovaj alat pretvara tekst u brojeve na osnovu značaja reči (TF-IDF).
- `MinMaxScaler` za kolonu `review_length` – ovaj alat svodi sve vrednosti na raspon između 0 i 1.
- Sve ove transformacije grupisaćemo u okviru jednog objekta – `ColumnTransformer` – koji spaja sve podatke u jedinstvenu numeričku matricu spremnu za treniranje.

Skript izgleda ovako:

```
1 from sklearn.preprocessing import MinMaxScaler
2 from sklearn.pipeline import Pipeline
3 from sklearn.compose import ColumnTransformer
4
5
6 # Preprocessor: TF-IDF for text, MinMaxScaler for numeric feature
7 preprocessor = ColumnTransformer(
8     transformers=[
9         ("title", TfidfVectorizer(), "review_title"),
10        ("text", TfidfVectorizer(), "review_text"),
11        ("length", MinMaxScaler(), ["review_length"])
12    ]
13 )
```

Šta je novina u ovom kodu?

U ovom kodu koristimo dve nove, ali veoma korisne komponente iz biblioteke scikit-learn.

1. ColumnTransformer – zadužen za obradu kolona

Omogućava nam da:

- jednu kolonu obradimo kao tekst, koristeći TfidfVectorizer;
 - a drugu kao broj, koristeći MinMaxScaler (ili neku drugu tehniku).
- Bez ColumnTransformera bismo morali ručno da obrađujemo svaku kolonu posebno i da ih sami spajamo u jedan skup. Ovo bi bilo komplikovano i podložno greškama.

2. MinMaxScaler – zadužen za skaliranje brojeva

Koristi se da bi vrednosti u numeričkoj koloni (review_length) bile uvek u istom opsegu, bez obzira na to kolike su.

Na primer:

- recenzija od 40 karaktera postaje 0.2;
- recenzija od 80 karaktera postaje 0.4;
- recenzija od 200 karaktera postaje 1.0; sve vrednosti su sada između 0 i 1.

Ukratko, zašto sve ovo radimo:

- Zašto TF-IDF?
Da model može da razume tekst.
- Zašto MinMaxScaler?
Da dužina recenzije ne zavlada rezultatima – i da model može na lakši način da interpretira takve vrednosti.

Pogledajmo zajedno kako se obavlja pretvaranje podataka u oblik prilagođen modelima mašinskog učenja:

<https://www.youtube.com/watch?v=wmU3283vpkk>

Video: Pretvaranje podataka u oblik prilagođen modelima mašinskog učenja

Autor: ITAcademy

Mini-aktivnost: Probaj samostalno - Od reči do broja

Uzmi manji podskup recenzija i napravi svoju TF-IDF transformaciju na review_title.

Igraj se sa parametrima (npr. ngram_range) i uporedi rezultate – šta se menja?

Ko je ovde najbolji? Definisanje, treniranje i evaluacija modela

Google Colab chat, #ml-projekat-reviews-v.2

Alex: Ljudi, imam tremu. Sledi ono zbog čega je ceo projekat uzbudljiv – stavljamo sve algoritme „u ring” i

gledamo koji se najbolje snalazi sa našim recenzijama! Imate li svog favorita?

Joakim: Znači, pravi ML rvački spektakl! Stižu kokice!

Marc: Samo da SVM ovaj put ne zabode na dužim recenzijama kao prošli put...

U ovom, poslednjem koraku, konačno koristimo sve što smo do sada pripremili:

- ulazne podatke koje smo prethodno transformisali (tekst i brojeve);
- više algoritama mašinskog učenja koje ćemo testirati;
- i jednostavan način da uporedimo njihove rezultate.

Šta radimo u ovoj fazi?

Sada prvi put koristimo pipeline – cevovod kroz koji prolaze svi naši podaci.

1. Prvo tekst i brojevi ulaze u obradu (TF-IDF, skaliranje...).
2. Onda različiti modeli pokušavaju da nauče obrasce iz podataka.
3. Na kraju, svaki model pokazuje svoje rezultate – i sve to elegantno u par linija koda!

Primer koda:

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.naive_bayes import MultinomialNB
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.svm import LinearSVC
6 from sklearn.metrics import classification_report
7
8
9 # List of classifiers
10 models = {
11     "Logistic Regression": LogisticRegression(max_iter=1000),
12     "Naive Bayes": MultinomialNB(),
13     "Decision Tree": DecisionTreeClassifier(),
14     "Random Forest": RandomForestClassifier(),
15     "Support Vector Machine": LinearSVC()
16 }
17
18 # Train and evaluate
19 for name, model in models.items():
20     print(f"\n
```

Marc: Sećate se kako smo nekad ručno obrađivali svaki korak? E pa sada sve ide kroz pipeline – i ništa ne može da se zaboravi ili preskoči!

Šta se ovde dešava?

- Za svaki model:
 - prvo se izvršava obrada teksta i brojeva (preprocessing);
 - zatim se model trenira (fit);
 - i na kraju, testira na podacima koje nije video (predict).
- `classification_report` nam daje metrike kao što su preciznost, odziv i F1 mera – sve ono što smo naučili ranije.

Šta je pipeline?

Pipeline je cevovod kroz koji podaci prolaze i u kome se definišu koraci obrade podataka.

1. Najpre se podaci transformišu (npr. tekst se pretvara u brojeve, brojevi se skaliraju).
2. Zatim se koristi algoritam koji na osnovu tih brojeva uči šablone.
3. Na kraju, dobijeni model se koristi za pravljenje predikcija.

Time smo sve što je ranije bilo razdvojeno (transformacija podataka + treniranje modela) spojili u jednu celinu.

Prednosti pipelinea:

- kod je čist i jednostavan;
- nema potrebe za ručnim pozivima `fit_transform`, `transform` itd.;
- sve se automatski izvršava pravim redosledom;
- lako se menja model ili dodaju novi koraci.

	precision	recall	f1-score	support
Logistic Regression				
negative	0.88	0.92	0.90	8932
neutral	0.72	0.64	0.68	2736
positive	0.94	0.97	0.97	21524
accuracy			0.92	33192
macro avg	0.88	0.81	0.85	33192
weighted avg	0.92	0.92	0.92	33192
Random Forest				
negative	0.89	0.94	0.92	8932
neutral	0.79	0.61	0.69	2736
positive	0.94	0.96	0.97	21524
accuracy			0.92	33192
macro avg	0.88	0.85	0.92	33192
weighted avg	0.93	0.93	0.92	33192
Naive Bayes				
negative	0.88	0.89	0.87	8932
neutral	0.81	0.28	0.42	2736
positive	0.92	0.98	0.96	21524
accuracy			0.90	33192
macro avg	0.84	0.72	0.75	33192
weighted avg	0.88	0.90	0.88	33192
Support Vector Machine				
negative	0.88	0.93	0.91	8932
neutral	0.74	0.64	0.69	2736
positive	0.96	0.97	0.97	21524
accuracy			0.92	33192
macro avg	0.86	0.85	0.92	33192
weighted avg	0.92	0.93	0.92	33192
Decision Tree				
negative	0.88	0.90	0.89	8932
neutral	0.82	0.58	0.68	2736
positive	0.94	0.95	0.95	21524
accuracy			0.91	33192
macro avg	0.82	0.81	0.81	33192
weighted avg	0.91	0.91	0.91	33192

Rezultati koje su naši modeli ostvarili

Alex: Okej, ovo je bilo napeto! Meni brojevi deluju vrlo ujednačeno. Jedino je Naive Bayes malo podbacio na neutralnim recenzijama.

Marc: Da, i meni je upalo u oči da modeli imaju najviše problema sa neutralnim recenzijama – sigurno zato što u našem skupu podataka upravo ima najmanje takvih recenzija.

Joakim: Logistic Regression daje tradicionalno dobre rezultate, ali opet blagu prednost imaju složeniji algoritmi – Random Forest i Support Vector Machine. Ipak, oni su resursno mnogo zahtevniji.

Alex: Znači, nema jednog pobednika, već biramo ono što najviše odgovara našim potrebama. Ja bih Random Forest pustila u produkciju, ali bih Logistic Regression zadržala za slučaj kada nemamo mnogo vremena i želimo da treniranje prođe brzo.

Marc: Slažem se. A možemo napraviti i „backup“ model, pa ih upoređivati ih u realnom radu.

Pogledajmo zajedno kako se obavljaju definisanje, treniranje i evaluacija modela:

<https://www.youtube.com/watch?v=LtCcY5JK4AM>

Video: Definisanje, treniranje i evaluacija modela
Autor: ITAcademy

Mini-aktivnost: Koji je model najbolji?

Trenirali smo pet različitih modela mašinskog učenja na istom skupu podataka. Svi modeli su trenirani na identičan način i testirani na istim recenzijama koje nisu bile uključene u trening. Na osnovu `classification_report` izlaza za svaki od modela, odgovorite na sledeća pitanja:

- Koji model ima najbolji rezultat za positive?
- Koji je najbolji za negative?
- Koji najviše „pokriva“ neutralne recenzije?
- Koji ima najbolju ukupnu F1 meru?
- Da li su razlike među modelima značajne ili su slični?
- Da ste na mestu Alex – koji biste model izabrali za produkciju i zašto?

Važno: U praksi, najbolji model ne znači uvek apsolutno najbolji broj, već i da je dovoljno brz, stabilan i lako za održavanje u stvarnom okruženju. Zato je uvek dobro diskutovati i o širim razlozima za izbor.

Diskutujte u timu ili napišite svoje razmišljanje na EduWall. Ako postoji razlika u metrikama, pokušajte da protumačite zašto je jedan model bolji od drugog – možda zbog toga što bolje hvata dugačke recenzije ili je stabilniji na balansiranom skupu podataka.

Zaključak lekcije

U ovoj lekciji napravili smo veliki korak – iz sveta pripreme i analize podataka prešli smo u arenu gde se algoritmi takmiče za titulu najboljeg modela!

Testirali smo više različitih pristupa, merili njihove rezultate i naučili kako konkretne metrike mogu otkriti nijanse koje obična tačnost ne može da pokaže.

Šta smo postigli u ovoj lekciji?

- Pripremili smo podatke za treniranje i testiranje modela.
- Implementirali smo više algoritama koristeći efikasn timer pristup.
- Izvršili smo poređenje i evaluaciju svih kandidata koristeći standardne metrike (preciznost, odziv, F1, tačnost).
- Dobili smo jasnu sliku o tome koji model najbolje odgovara našem konkretnom problemu klasifikacije sentimenta.

Takeaways

- Nema univerzalnog pobednika: Svaki model ima svoje prednosti i slabosti. Važno je razumeti ne samo ko je najbolji, već i zašto.
- Metrike su naš kompas: Osim tačnosti, vredne su pažnje i preciznost, odziv i F1, jer otkrivaju kako model razume svaku klasu ponaosob.
- Pipeline = profesionalizam: Korišćenje pipelinea pojednostavljuje kod, smanjuje greške i omogućava brzu izmenu ili skaliranje celog toka rada.
- Diskusija u timu vodi do boljih odluka: Pravi šampion nije onaj koji pobeđi na papiru, već onaj koji se uklapa u poslovne potrebe i ograničenja projekta.

Sada ste postavili temelje profesionalnog rada u mašinskom učenju. Razumevanje izbora i evaluacije modela čini vas nečim više od korisnika gotovih rešenja: pravim graditeljem pametnih sistema!

Prava vrednost je u tome što sada znate kako i zašto pravite izbor – i spremni ste da argumentovano branite svoja rešenja pred timom i pred poslodavcem.

Sledeći korak: Odabir najboljeg algoritma za treniranje modela

Sad kad imamo pobednički model, vreme je da ga sačuvamo – ali i da naučimo kako ga koristiti izvan ovog projekta. U sledećoj lekciji, naučićeš kako se pravi pravi model za poneti: kako ga eksportovati, deliti sa timom i integrisati u druge aplikacije ili radne tokove.

Praktično, izlazimo iz laboratorije – tvoj model postaje alat spreman za realan svet!

Jesmo li spremni da ga pustimo u promet? Krenimo dalje!